

Lab 2: Simulation and Robust Inference

SOCIOL 723, Social Statistics II, Thursday, September 3

Wenhao Jiang

Table of contents

1	Goals	1
2	Building a sampling distribution	2
3	Robust standard errors on real data	4
3.1	By hand	4
3.2	HC0 through HC3	4
3.3	The convenience tool: <code>stargazer</code>	5
4	Clustering	6
4.1	By hand: sum within clusters first, then square	6
4.2	Which level for the GSS?	7
5	The bootstrap	8
5.1	By hand	8
5.2	With the <code>boot</code> package	9
5.3	The cluster bootstrap	10
6	A convenient shortcut: <code>fixest</code>	10
7	Exercises	11
8	Session information	11

1 Goals

1. See a sampling distribution by *building* one, through simulation.
2. Watch classical standard errors fail under heteroskedasticity, and watch robust standard errors fix them.
3. Compute HC0–HC3 and cluster-robust standard errors in R, by hand and with packages.
4. Bootstrap, from first principles and with `boot`, including the cluster bootstrap.

```

library(dplyr)
library(ggplot2)
library(sandwich)      # vcovHC(), vcovCL()
library(lmtest)        # coeftest()
library(boot)          # bootstrap
library(fixest)        # fast fixed effects with clustered SEs

gss <- readRDS("../Data/gss_earnings.rds")
c(n = nrow(gss), psu = n_distinct(gss$psu),
  stratum = n_distinct(gss$stratum), region = n_distinct(gss$region))
#>      n      psu stratum  region
#>  3509     839    454      9

```

2 Building a sampling distribution

The fastest way to understand a standard error is to construct one.

```

set.seed(723)

## a population we control completely
sim_once <- function(n = 200, beta1 = 0.5, hetero = FALSE) {
  x <- rnorm(n, mean = 0, sd = 1)
  sd_i <- if (hetero) 0.5 * exp(0.8 * x) else 1      # error sd
  e <- rnorm(n, sd = sd_i)
  y <- 1 + beta1 * x + e
  m <- lm(y ~ x)
  c(b = coef(m)[2],
    se_classical = sqrt(diag(vcov(m)))[2],
    se_hc3 = sqrt(diag(vcovHC(m, type = "HC3")))[2])
}

reps_homo <- t(replicate(4000, sim_once(hetero = FALSE)))
head(round(reps_homo, 4), 3)
#>      b.x se_classical.x se_hc3.x
#> [1,] 0.4439      0.0671  0.0649
#> [2,] 0.5575      0.0770  0.0782
#> [3,] 0.5671      0.0789  0.0752

tibble::tibble(b = reps_homo[, 1]) |>
  ggplot(aes(b)) +
  geom_histogram(bins = 50, fill = "navy", alpha = .65) +
  geom_vline(xintercept = 0.5, colour = "firebrick", linewidth = .7) +
  labs(x = expression(hat(beta)[1]), y = "count") +

```

```
theme_minimal(base_size = 10)
```

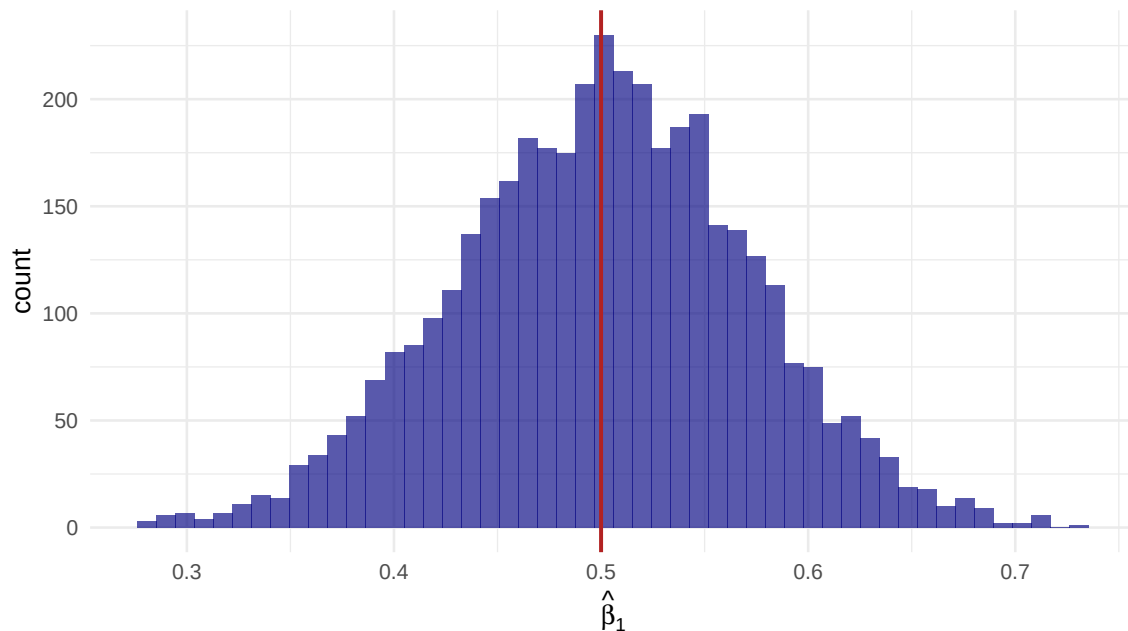


Figure 1: Sampling distribution of $\hat{\beta}_1$ over 4,000 samples of size 200. The true value is 0.5.

The *true* standard error is the standard deviation of this histogram. Compare it to what the two formulas report on average:

```
c(truth = sd(reps_homo[, "b.x"]),
  mean_classical = mean(reps_homo[, "se_classical.x"]),
  mean_hc3 = mean(reps_homo[, "se_hc3.x"]))
#>      truth mean_classical      mean_hc3
#>      0.07095      0.07102      0.07165
```

Under homoskedasticity both are right; the robust one is very slightly noisier. That is the small price we pay for insurance. Now turn heteroskedasticity on:

```
reps_het <- t(replicate(4000, sim_once(hetero = TRUE)))
c(truth = sd(reps_het[, "b.x"]),
  mean_classical = mean(reps_het[, "se_classical.x"]),
  mean_hc3 = mean(reps_het[, "se_hc3.x"]))
#>      truth mean_classical      mean_hc3
#>      0.12349      0.06559      0.11578
```

The classical formula is badly wrong; HC3 tracks the truth.

3 Robust standard errors on real data

3.1 By hand

The scalar formula from lecture, for the bivariate case:

$$\hat{V}_{HC0}(\hat{\beta}_1) = \frac{\sum_i d_i^2 \hat{e}_i^2}{(\sum_i d_i^2)^2}, \quad d_i = X_i - \bar{X}.$$

```
m_biv <- lm(learn ~ educ, data = gss)
d <- gss$educ - mean(gss$educ)
eh <- residuals(m_biv)

se_hc0_manual <- sqrt(sum(d^2 * eh^2) / (sum(d^2))^2)
se_hc0_pkg <- sqrt(diag(vcovHC(m_biv, type = "HC0")))[ "educ" ]
se_classical <- sqrt(diag(vcov(m_biv)))[ "educ" ]

round(c(classical = se_classical, hc0_manual = se_hc0_manual,
        hc0_sandwich = se_hc0_pkg), 6)
#>      classical.educ      hc0_manual hc0_sandwich.educ
#>           0.005213           0.005416           0.005416
```

The general matrix version, also by hand:

```
m <- lm(learn ~ educ + exper + I(exper^2) + female + black, data = gss)
X <- model.matrix(m); e <- residuals(m)
bread <- solve(crossprod(X))
meat <- crossprod(X * e)      # = sum_i e_i^2 x_i x_i'
V_hc0 <- bread %*% meat %*% bread
round(cbind(manual = sqrt(diag(V_hc0)),
            sandwich = sqrt(diag(vcovHC(m, type = "HC0")))), 6)
#>              manual sandwich
#> (Intercept) 0.105397 0.105397
#> educ        0.005475 0.005475
#> exper       0.005696 0.005696
#> I(exper^2)  0.000118 0.000118
#> female      0.028126 0.028126
#> black       0.038527 0.038527
```

3.2 HC0 through HC3

```
ses <- sapply(c("const", "HC0", "HC1", "HC2", "HC3"),
             function(ty) sqrt(diag(vcovHC(m, type = ty))))
round(ses, 5)
#>           const      HC0      HC1      HC2      HC3
```

```
#> (Intercept) 0.09903 0.10540 0.10549 0.10556 0.10572
#> educ        0.00510 0.00547 0.00548 0.00548 0.00549
#> exper       0.00546 0.00570 0.00570 0.00571 0.00572
#> I(exper^2)  0.00011 0.00012 0.00012 0.00012 0.00012
#> female      0.02793 0.02813 0.02815 0.02815 0.02818
#> black       0.03856 0.03853 0.03856 0.03858 0.03864
```

```
coeftest(m, vcov = vcovHC(m, type = "HC3"))
#>
#> t test of coefficients:
#>
#>              Estimate Std. Error t value      Pr(>|t|)
#> (Intercept)  7.390108   0.105715   69.91 < 0.0000000000000002 ***
#> educ         0.140532   0.005490   25.60 < 0.0000000000000002 ***
#> exper        0.052875   0.005716    9.25 < 0.0000000000000002 ***
#> I(exper^2)   -0.000739   0.000118   -6.26    0.000000000044 ***
#> female      -0.375785   0.028177  -13.34 < 0.0000000000000002 ***
#> black       -0.191473   0.038636   -4.96    0.00000075431 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

With $n = 3509$ the four robust variants are nearly identical. Rerun this on a subsample of 100 and the gaps open up:

```
set.seed(1)
small <- gss[sample(nrow(gss), 100), ]
m_small <- lm(learn ~ educ + exper + I(exper^2) + female + black, data = small)
round(sapply(c("const", "HC0", "HC1", "HC2", "HC3"),
             function(ty) sqrt(diag(vcovHC(m_small, type = ty))["educ"])), 5)
#> const.educ  HC0.educ  HC1.educ  HC2.educ  HC3.educ
#>    0.02291    0.02291    0.02363    0.02434    0.02600
```

3.3 The convenience tool: stargazer

For a paper-style table, hand **stargazer** the model once per column and a list of standard errors; it recomputes the stars from whatever SEs you supply.

```
library(stargazer)
se_hc1 <- sqrt(diag(vcovHC(m, type = "HC1")))
se_hc3 <- sqrt(diag(vcovHC(m, type = "HC3")))
stargazer(m, m, m,
          se = list(NULL, se_hc1, se_hc3),
          column.labels = c("Classical", "HC1", "HC3"),
          keep = c("educ", "female", "black"),
```

```

keep.stat = c("n", "rsq"),
digits = 4,
type = "text", no.space = TRUE)

#>
#> =====
#>                Dependent variable:
#>                -----
#>                        llearn
#>            Classical      HC1      HC3
#>              (1)        (2)        (3)
#> -----
#> educ            0.1405***  0.1405***  0.1405***
#>                  (0.0051)  (0.0055)  (0.0055)
#> female          -0.3758*** -0.3758*** -0.3758***
#>                  (0.0279)  (0.0282)  (0.0282)
#> black           -0.1915*** -0.1915*** -0.1915***
#>                  (0.0386)  (0.0386)  (0.0386)
#> -----
#> Observations    3,509      3,509      3,509
#> R2              0.2326      0.2326      0.2326
#> =====
#> Note:           *p<0.1; **p<0.05; ***p<0.01

```

Same coefficients in every column, only the standard errors move: the whole lecture in one table. For a paper, switch `type = "text"` to `type = "latex"` and always say in the table note which variance estimator each column uses.

4 Clustering

The GSS is a multi-stage area probability sample: it draws primary sampling units (**psu**) within strata (**stratum**), then households within PSUs. Respondents in the same PSU live in the same neighbourhood, so their errors are correlated.

4.1 By hand: sum within clusters first, then square

Lecture's boxed scalar formula, plus the finite-sample scaling software applies by default (the cluster analogue of HC1's degrees-of-freedom correction; Stata's default too):

$$\hat{V}_{\text{CR1}}(\hat{\beta}_1) = \underbrace{\frac{G}{G-1} \cdot \frac{n-1}{n-k}}_c \frac{\sum_g S_g^2}{(\sum_i d_i^2)^2}, \quad S_g = \sum_{i \in g} d_i \hat{e}_i.$$

```

Sg <- tapply(d * eh, gss$psu, sum) # one number per cluster
G <- n_distinct(gss$psu); n <- nrow(gss); k <- 2
c1 <- G / (G - 1) * (n - 1) / (n - k)
se_cr_manual <- sqrt(c1 * sum(Sg^2) / (sum(d^2))^2)
se_cr_pkg <- sqrt(vcovCL(m_biv, cluster = ~ psu)["educ", "educ"])
round(c(scaling = c1, cr_manual = se_cr_manual,
        cr_sandwich = se_cr_pkg, hc0 = se_hc0_manual), 6)
#>      scaling      cr_manual cr_sandwich      hc0
#>      1.001479      0.005610      0.005610      0.005416

```

Sum within clusters first, then square. With 839 clusters the scaling c is negligible; it matters when clusters are few. Up to that factor, one observation per cluster collapses this to the robust formula: the two are one idea.

4.2 Which level for the GSS?

The GSS is a multi-stage sample: PSUs are the *sampled* units, and no regressor in this model varies at any group level. So the design says **cluster on psu**, full stop. Coarser levels would allow extra dependence the design gives no reason to expect, at the price of a noisier variance estimate.

```

se_psu <- sqrt(diag(vcovCL(m, cluster = ~ psu)))
stargazer(m, m, m,
  se = list(NULL, se_hc3, se_psu),
  column.labels = c("Classical", "HC3", "Cluster PSU"),
  keep = c("educ", "female", "black"),
  keep.stat = c("n", "rsq"),
  type = "text", no.space = TRUE)
#>
#> =====
#>
#>                Dependent variable:
#>                -----
#>                        lnearn
#>            Classical      HC3      Cluster PSU
#>                (1)        (2)        (3)
#> -----
#> educ            0.141***   0.141***   0.141***
#>                (0.005)   (0.005)   (0.006)
#> female          -0.376***  -0.376***  -0.376***
#>                (0.028)   (0.028)   (0.027)
#> black           -0.191***  -0.191***  -0.191***
#>                (0.039)   (0.039)   (0.040)
#> -----
#> Observations    3,509      3,509      3,509

```

```
#> R2          0.233      0.233      0.233
#> =====
#> Note:          *p<0.1; **p<0.05; ***p<0.01
```

Clustering moves the standard errors only modestly here: GSS PSUs hold few respondents each, and `educ` varies mostly *within* PSUs, so the surviving cross terms are few and small.

5 The bootstrap

5.1 By hand

```
set.seed(723)
B <- 2000
boot_b <- replicate(B, {
  idx <- sample(nrow(gss), replace = TRUE)
  coef(lm(lnearn ~ educ + exper + I(exper^2) + female + black,
    data = gss[idx, ]))["educ"]
})

c(estimate = coef(m)["educ"],
  se_boot = sd(boot_b),
  se_hc3 = sqrt(diag(vcovHC(m, type = "HC3"))))["educ"])
#> estimate.educ      se_boot      se_hc3.educ
#>      0.140532      0.005508      0.005490

quantile(boot_b, c(.025, .975))      # percentile interval
#>      2.5%      97.5%
#> 0.1300 0.1515
```

The pairs bootstrap resamples (Y_i, X_i) jointly, so it makes no assumption about σ_i^2 . Asymptotically it targets the same sandwich variance that the HC estimators do, which is why it lands in the same neighbourhood, the small gaps among HC0/HC3/bootstrap here are finite-sample, not conceptual.

```
tibble::tibble(b = boot_b) |>
  ggplot(aes(b)) +
  geom_histogram(bins = 50, fill = "navy", alpha = .65) +
  geom_vline(xintercept = coef(m)["educ"], colour = "firebrick") +
  labs(x = "bootstrap replicate of the coefficient on educ", y = "count") +
  theme_minimal(base_size = 10)
```

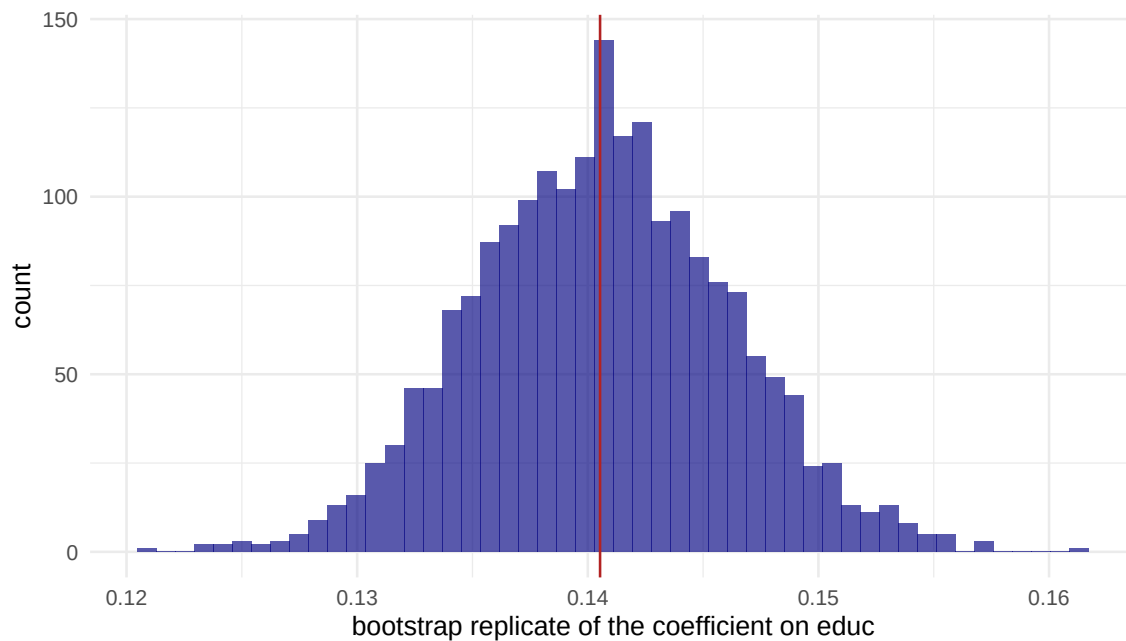



Figure 2: Bootstrap distribution of the coefficient on years of schooling, $B = 2,000$.

The histogram is bell-shaped here, so the normal-approximation and percentile intervals will nearly coincide; when the histogram is skewed or lumpy, they disagree, and the shape is the reason to prefer the percentile side.

5.2 With the boot package

```
b_fun <- function(data, idx) {
  unname(coef(lm(learn ~ educ + exper + I(exper^2) + female + black,
    data = data[idx, ]))["educ"])
}
set.seed(723)
bo <- boot(gss, b_fun, R = 2000)
boot.ci(bo, type = c("norm", "perc"))
#> BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
#> Based on 2000 bootstrap replicates
#>
#> CALL :
#> boot.ci(boot.out = bo, type = c("norm", "perc"))
#>
#> Intervals :
#> Level      Normal          Percentile
#> 95%    ( 0.1296, 0.1513 )  ( 0.1296, 0.1512 )
#> Calculations and Intervals on Original Scale
```

5.3 The cluster bootstrap

Resample **clusters**, not individuals:

```
set.seed(723)
psus <- unique(gss$psu)
boot_cl <- replicate(1000, {
  draw <- sample(psus, length(psus), replace = TRUE)
  dat <- do.call(rbind, lapply(draw, function(p) gss[gss$psu == p, ]))
  coef(lm(lnearn ~ educ + exper + I(exper^2) + female + black,
    data = dat))["educ"])
})
c(cluster_boot = sd(boot_cl),
  analytic_CR1 = sqrt(diag(vcovCL(m, cluster = ~ psu)))["educ"])
#>      cluster_boot analytic_CR1.educ
#>      0.005585      0.005611
```

Warning

If you had resampled *individuals* here, the bootstrap would have reproduced the too-small non-clustered standard error just as faithfully as the analytic formula does. The bootstrap does not know your design; you have to tell it.

6 A convenient shortcut: `fixest`

For everyday work, `fixest::feols()` computes clustered standard errors on the fly and is very fast.

```
fe <- feols(lnearn ~ educ + exper + I(exper^2) + female + black | year,
  data = gss, cluster = ~ psu)
summary(fe)
#> OLS estimation, Dep. Var.: lnearn
#> Observations: 3,509
#> Fixed-effects: year: 6
#> Standard-errors: Clustered (psu)
#>
#>      Estimate Std. Error t value      Pr(>|t|)
#> educ      0.138413   0.005593  24.745    < 2.2e-16 ***
#> exper      0.053594   0.006032   8.885    < 2.2e-16 ***
#> I(I(exper^2)) -0.000752   0.000123  -6.111 0.0000000015165 ***
#> female     -0.373403   0.026991 -13.834    < 2.2e-16 ***
#> black     -0.195537   0.040129  -4.873 0.0000013160266 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#> RMSE: 0.813625      Adj. R2: 0.239282
```

```
#> Within R2: 0.231064
```

The `| year` term absorbs year fixed effects (FWL under the hood, as in Lab 1), and `cluster = ~ psu` supplies the cluster-robust variance.

7 Exercises

1. **Coverage of the classical interval.** Repeat the coverage simulation with error standard deviation $0.5 \exp(1.5x_i)$ instead of $0.5 \exp(0.8x_i)$. How bad does classical coverage get? Does HC3 hold up?
2. **HC3 versus HC0 in small samples.** For $n \in \{30, 60, 120, 500\}$, simulate 2,000 heteroskedastic datasets and record the coverage of intervals built from HC0 and from HC3. At what n does the difference stop mattering?
3. **Choosing the cluster level.** Suppose you wanted to estimate the effect of a state-level minimum-wage law on individual earnings, using individual GSS data. At what level would you cluster, and why? What if the law varied by state *and* year?
4. **A statistic with no easy formula.** Estimate the experience profile's turning point $-\hat{\beta}_{\text{exper}}/(2\hat{\beta}_{\text{exper}^2})$ and bootstrap it ($B = 5,000$). Plot the histogram of the replicates: is it bell-shaped? Report both the normal-approximation and the percentile interval, and say which you would report and why.

8 Session information

```
sessionInfo()
#> R version 4.5.2 (2025-10-31)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS Tahoe 26.6.2
#>
#> Matrix products: default
#> BLAS:      /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/Libraries/libBLAS.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods   base
#>
```

```

#> other attached packages:
#> [1] stargazer_5.2.3 fixest_0.13.2 boot_1.3-32 lmtest_0.9-40
#> [5] zoo_1.8-15 sandwich_3.1-1 ggplot2_4.0.2 dplyr_1.2.0
#>
#> loaded via a namespace (and not attached):
#> [1] gtable_0.3.6 jsonlite_2.0.0 compiler_4.5.2
#> [4] tinytex_0.58 Rcpp_1.1.1 tidyselect_1.2.1
#> [7] scales_1.4.0 yaml_2.3.12 fastmap_1.2.0
#> [10] lattice_0.22-7 R6_2.6.1 labeling_0.4.3
#> [13] generics_0.1.4 Formula_1.2-5 knitr_1.51
#> [16] tibble_3.3.1 pillar_1.11.1 RColorBrewer_1.1-3
#> [19] rlang_1.1.7 xfun_0.56 stringmagic_1.2.0
#> [22] S7_0.2.1 otl_0.2.0 cli_3.6.5
#> [25] withr_3.0.2 magrittr_2.0.4 digest_0.6.39
#> [28] grid_4.5.2 nlme_3.1-168 lifecycle_1.0.5
#> [31] vctrs_0.7.1 evaluate_1.0.5 glue_1.8.0
#> [34] numDeriv_2016.8-1.1 farver_2.1.2 dreamerr_1.5.0
#> [37] rmarkdown_2.30 tools_4.5.2 pkgconfig_2.0.3
#> [40] htmltools_0.5.9

```